Lightstep version **1.0**

# Programming Guide

Third Edition

AIOI·SYSTEMS CO.,LTD.

**Revision History**

| Date | Edition | Detail |
|---|---|---|
| 2012/6/29 | First Edition | |
| 2013/12/25 | Second Edition | Changed the support environment<br>Added a description on changes made in ver.1.0.0.3. |
| 2015/12/18 | Third Edition | Changed the support environment<br>Added a description on changes made in ver.1.0.0.4. |

# Contents


1. Introduction ..... 1

2. Mechanism of Lightstep ..... 4

2.1 Basic Concept ..... 4

2.2 Configuration File ..... 4

2.3 Class Configuration ..... 4

2.4 How Commands are Sent and Received ..... 5

3. How to Use - Basics ..... 7

3.1 Add into the Project ..... 7

3.2 Create an Instance ..... 8

3.3 Set up the License Key ..... 9

3.4 Connect to the Controller ..... 9

3.5 Send a Command ..... 9

3.6 Receive Command from the Controller ..... 10

3.7 Disconnect from the Controller ..... 11

3.8 Check the Command Log ..... 11

4. How to Use and Put into Practice ..... 12

4.1 Create an Instance ..... 12

4.2 Connect with the Controller ..... 14

4.3 Communicate by Using a Synchronous Method ..... 15

4.3.1 What is Synchronous Communication ..... 15
4.3.2 List of Synchronous Method ..... 15
4.3.3 Advantages and Disadvantages ..... 15
4.3.4 Implement Synchronous Connection ..... 16
4.3.5 Implement Synchronous Transmission ..... 17

4.4 Communicate by Using the Asynchronous Method ..... 18

4.4.1 Asynchronous Communication ..... 18
4.4.2 List of the Asynchronous Method ..... 18
4.4.3 Advantages and Disadvantages ..... 19
4.4.4 Implement Asynchronous Connection ..... 19
4.4.5 Implement Asynchronous Transmission ..... 21
4.4.6 Implementation of Asynchronous Disconnection ..... 25
4.4.7 Using the Synchronous Method at the Same Time ..... 26

4.5 Commands Received from the Controller ..... 27

4.5.1 General Flow of Command Receiving Process ..... 27
4.5.2 Implementaiton Cases ..... 27

4.6 Use of the Command Log ..... 30

4.6.1 Overview of the Operation ..... 30
4.6.2 Location of the Saved Log File ..... 30
4.6.3 How to Read the Command ..... 31
4.6.4 Automatic Deletion of Log ..... 32
4.6.5 Change the Log Setting ..... 32

4.6.6 Stop the Log from Outside the Application ... 33
4.7 Processing Commands ... 33
4.7.1 Binary Conversion ... 33
4.7.2 Create Repeatable Commands ... 34
4.7.3 Analysis and Breakdown of the “t” Command ... 35
4.8 Error Processing ... 38
4.8.2 Errors from the Asynchronous Method ... 38
4.8.3 Errors Caused by the Internal Thread ... 39
4.8.4 Get Additional Error Information ... 39
4.8.5 Resend the Transmission Command ... 40
5. How to Use – Application Guide ... 42
5.1 Dealing with Various Types of Controllers ... 42
5.1.1 ControllerBase Class ... 42
5.1.2 Connection String ... 42
5.1.3 IPolling Interface ... 43
5.2 Get the Communication Log with the Host Application ... 44
5.3 Unique Functions of Each Controller Type ... 45
5.3.1 For the Ethernet Type ... 45
5.3.2 For the PCI Bus Type ... 45
5.3.3 For the RS-232C Type ... 45
6. Distribution of Application ... 46
6.1 Distribution Files ... 46
6.2 Placement Location ... 46
6.3 About Lightstep Version ... 46
7. Migrating from AiPlug ... 47
7.1 Determine the Migration Direction ... 47
7.2 Comparing with AiPlug ... 47
7.3 Main Points When Replacing ... 48
7.4 Change the Error Process ... 49

# 1. Introduction

### About this Manual

This manual explains how to use the library (Lightstep) to program communication with the controller. This manual explains the **Lightstep** functions and the method of using it.
For detailed specifications of the library, please refer to the “Lightstep API Reference” help file.

### What is Lightstep?

This library is a class library that runs on **.NET Framework 2.0**. When used, it facilitates the processing of sending and receiving commands and provides stable communication. It is a .NET library with similar functions as those of AiPlug. It is a successor to **AiPlug**.

### Main Features

- Can connect and disconnect with the controller and send/receive commands. The application developer can communicate with the controller by specifying only the parameters and command.
- Can connect and transmit command asynchronously. Developers can easily program applications using multithread functions.
- Because there is no need to arrange on the application screen, there is more leeway in how the application is incorporated.
- The DLL file can be distributed to the execution folder by coping and pasting.

### Operating Environment

| | |
|---|---|
| Runtime OS | 32bit: Windows 10 / 8.1 / 8 / 7 / Vista / XP /<br>Windows Server 2008 / 2003<br>64bit(*1): Windows 10 / 8.1 / 8 / 7 / Windows Server 2012 / 2008R2 |
| Development OS | 32bit: Windows 10 / 8.1 / 8 / 7 / Vista / XP<br>64bit(*1): Windows 10 / 8.1 / 8 / 7 |
| Development Environment | Microsoft Visual Studio 2015 / 2013 / 2012 / 2010 / 2008 / 2005<br>Windows Forms (VB/C#) |
| Compatible .NET Framework Version | 4.6 / 4.5 / 4.0 / 3.5 / 3.0 / 2.0 |
| Runtime Required for Execution | Microsoft .NET Framework 2.0 or later<br>※We recommend using the latest update programs of the .NET Framework. |

(*1) In the case of the PCI bus controller, the 64 bit version cannot be used. You need to use IoLibX when using the PCI bus controller.

### Installation in the Development Environment

To install, start up the setup file (”Lightstep**_setup_x**.msi”) and finish installing according to the screen. To run, you must have administrative privileges.

When installation is completed, files required will be copied to the specified destination. Also, Global Assembly Cache (GAC) will also be registered.

We recommend using the latest **.NET Framework** service pack.

### Uninstall from the Development Environment

Uninstall from the Windows control panel.

### License

One **Lightstep** license per 1 developer’s PC used for development.
There is no restriction on the number of PCs.

You may redistribute the execution program you developed by using **Lightstep**. The following files can be redistributed.

| |
|---|
| AioiSystems.Lightstep.dll<br>AioiSystems.Lightstep.resources.dll |

### Terms used in this Manual

| | |
|---|---|
| Host Application | Application software that uses **Lightstep** |
| Frame | The format that matches the protocol of the controller is processed for the command text.<br>Example: STX + command + ETX |
| Controller Class | Derived classes from **ControllerBase** class<br>**EthernetController** class, **PciBusController** class,<br>**Rs232cController** class |

### Changes made in each version

version 1.0.0.4

Corrected the problem that the completion of asynchronous sending may not be recognized correctly.

version 1.0.0.3

Under certain conditions, the following problems may have occurred but have since been corrected:

- A build error occurred after a control class was placed on the form in Visual Studio 2012.
- [PCI/RS232C] In a project executed by .NET Framework 4.x., if the PCI bus controller or RS232C controller was used, it took 500msec - 15sec for the CommandReceived event to occur, even for just 1 light module, after the CONFIRM button was pressed.
- [PCI/RS232C] During the automatic polling function, the automatic polling of the block did not occur for that block even after adding the block number for polling.

# 2. Mechanism of Lightstep

This section explains the structure of **Lightstep**.

## 2.1 Basic Concept

Because **Lightstep** is constructed from multiple classes, use the class that is appropriate for your application. **Lightstep** is a general term for those classes.

In **Lightstep**, communication objects are available for various types of controllers, such as TCP/IP, PCI-Bus, or RS-232C. These objects provide the methods to connect/disconnect or send/receive commands with the controller.

## 2.2 Configuration File

The configuration of Lightstep is the following.

| | |
|---|---|
| AioiSystems.Lightstep.dll | DLL of Lightstep |
| AioiSystems.Lightstep.XML | API's help data used by Visual Studio, e.g., Intellisense. Not needed at runtime. |
| AioiSystems.Lightstep.resources.dll | Resource file for Japanese error messages. It is in the "**ja**" folder. |

## 2.3 Class Configuration

Each class of Lightstep is stored in the namespace "**AioiSystems.Lightstep**". The main classes are the following.

| | |
|---|---|
| EthernetController Class | Ethernet interface controller communication class |
| PciBusController Class | PCI bus type of controller communication class |
| Rs232cController Class | RS-232C type of controller communication class |
| ControllerBase Class | Abstract class of the above 3 classes |

## 2.4 How Commands are Sent and Received

**Lightstep** processes commands that are sent and received. The procedure will be explained below.

**Send a Command**

When the application sends a command to the controller, the process of **Lightstep** is explained below.

(1) From the application (expressed as Host Application below) that uses **Lightstep**, a command is passed to **Lightstep** by using the "**SendCommand**" method.

The command here refers to the actual command such as "**Z**" and "**P100000112345**". Start text (**STX**) and end text (**ETX**) are excluded.




(2) **Lightstep** processes the format that matches the communicating controller (below shows the status of the processed format expressed as **frame**).

(Example) For Ethernet Controller

Z → (STX)(Sequence Number)(Command Length)+Z+(ETX)

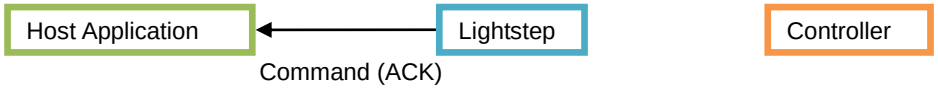
(3) Send the frame to the controller through the communication method that the controller can accept.




(4) Wait for the response command (ACK) from the controller.




(5) The ACK command portion (excluding the frame portion) is passed to the host application.



**Receive a Command**

Below is an application of how Lightstep processes a command(Note1) sent from the controller until the host application receives that command.

**(Note 1)** This is the command sent from the controller (the "**t**" command), not the response command.

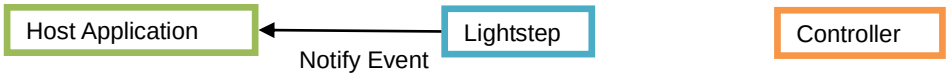
(1) The "**t**" command is sent from the controller.



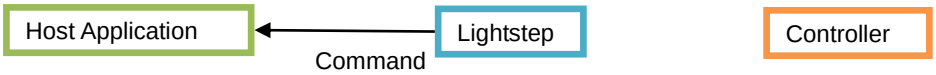

(2) Lightstep stores the command that it received in the internal buffer and sends a response (ACK) to the controller. (When the class of PcIBusController or Rs232cController is used, the "**StartPolling**" method must be executed.)




(3) Lightstep issues the "**CommandReceived**" event to notify the host application that it has received the command from the controller. However, this does not mean that the command is passed through this event.




(4) The host application complements the event. By using the "**GetCommand**" method, it gets the command that Lightstep has stored. Get only the command, not the frame.




After the host application command is sent and while waiting for the ACK response, sometimes a command from the controller is received. If that happens, the process (2) (sending of ACK) is immediately performed and the process (3) (event occurrence) is executed with a new thread. The ACK reception processing for the command sent by the host application will then resume.

Thus, Lightstep performs frame processing according to the controller's communication method and sends commands as well as checks the ACK response. After performing all these functions, it returns control to the host application. Also, as for receiving commands, it receives and sends ACK and notifies the timing by way of event and passes the command text that facilitates application process.

# 3. How to Use - Basics

In this section, the basics of how to use Lightstep will be explained. The goal is to provide a basic understanding. For this reason, not all necessary information is presented in this section. Make sure to read section 4 that discusses practical ways of using Lightstep.

## 3.1 Add into the Project

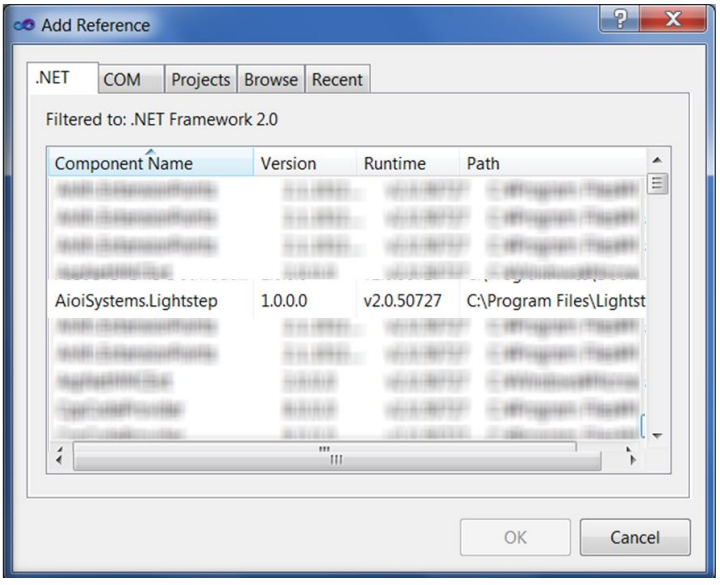
Below is an explanation on how to incorporate this library into the new project of Microsoft Visual Studio. Lightstep is a class library of **.NET Framework**. To incorporate, specify the DLL file from the reference setting.

**Procedure**

(1) In Visual Studio, open the project that uses Lightstep.

(2) From the menu "**Project**", select "**Add Reference . . .**" and display "**Add Reference**" screen.

(3) In the "**.NET**" tab, select "**AioiSystems.Lightstep**" and then press "**OK**".




※Version number may be different.

## 3.2 Create an Instance

To use Lightstep, you must first create an instance that matches the controller's communication method. The relationship between the controller's communication method and which class to use are shown below. These classes will be expressed as "**Conroller Class**".

| Controller Type | Class Used |
|---|---|
| Ethernet type of controller | EthernetController |
| PcI bus controller | PciBusController |
| RS-232C type controller | Rs232cController |

One controller is assigned with one instance. When multiple controllers are used, create only the number of instances for the number of controllers.

### Dynamically Created from the Code at Runtime

To create an instance, a variable is declared in the same way as the other class of **.NET Framework**. Make sure the variable is within the appropriate valid range.

**VB Code Example for the Ethernet Type**

```
Imports AioiSystems.Lightstep

Public Class Form1
    Private WithEvents _controller As New EthernetController
```

**C# Code Example for the Ethernet Type**

```
using AioiSystems.Lightstep;

namespace WindowsApplication1
{
    public partial class Form1 : Form
    {
        private EthernetController _controller = new EthernetController();
```

### Created on the Form Editor

Can be arranged as a component when designing a form.

### Procedure

(1) Select "**Choose Toolbox Items**" from Visual Studio's "**Tools**" menu.

(2) From the list of components displayed on the "**.NET Framework Component**" tab of the "**Choose Toolbox Items**" screen, put a check mark on the controller class that you want to use (refer to the previously mentioned "**Classes Used**"), then press "**OK**".

(3) Open "**Form Editor**" and "**Toolbox**" windows.

(4) The class that was added to the "**Toolbox**" is registered, so use a mouse to drag it on top of the form and arrange.

## 3.3 Set up the License Key

When a new instance of the controller class is created, you need to set a new license key for that instance. Use the "**SetLicense"** method of the controller class and specify the licence key to its argument either immediately after an instance is created or before connecting to the controller.

The license key is a 24-digit character set issued when Lightstep is purchased.

**VB Code Example**

```
_controller.SetLicense ("xxxx-xxxx-xxxx-xxxx-xxxx")
```

**C# Code Example**

```
_controller.SetLicense("xxxx-xxxx-xxxx-xxxx-xxxx");
```

If the software is executed without specifying a license key, it is recognized as a trial version. If it is a trial version, a dialog will be displayed when the controller is connected. When the "**OK**" button is pressed, the process resumes. All other functions and behavior are the same.

## 3.4 Connect to the Controller

To send and receive commands with the controller, the connection process must be performed. The "**Connect**" method of the controller class is used.

**VB Code Example for the Ethernet Type**

```
_controller.Connect("192.168.1.254", 5003)
```

**C# Code Example for the Ethernet Type**

```
_controller.Connect("192.168.1.254", 5003);
```

※**Connect** method has multiple overloads.

※If unable to connect, an exception will occur.

## 3.5 Send a Command

A command will be executed when the connection process was successful. The "**SendCommand**" method of the controller class is used.

**VB Code Example**

```
_controller.SendCommand("Z")
```

**C# Code Example**

```
_controller.SendCommand("Z");
```

※ **SendCommand** has multiple overloads.

※ Get the return value for ACK.

※ An exception is raised when there is timeout for a command, receives a negative response, or when an error occurs.

## 3.6 Receive Command from the Controller

When a command from the controller is sent ("**t**" command), a "**CommandReceived**" event occurs. Register the "**event handler**" ahead of time. The **event handler** is created by coding with **IDE**.

Below are examples of registration code.

**VB Code Example**

```
AddHandler _controller.CommandReceived, AddressOf _controller_CommandReceived
```

**C# Code Example**

```
_controller.CommandReceived += new EventHandler(_controller_CommandReceived);
```

The "**CommandReceived**" event only notifies the receipt of a command. Use the "**GetCommand**" method to get the command received,. The return value of this method is "**CommandInfo**" object, and with the "**GetCommandText**" method of the return value, you can get the command in character string type.

If you use the **PciBusController** or **Rs232cController** class, you have to execute the "**StartPolling**" method in order to receive the command. For more information, please refer to section 4 on practical examples.

### Implementation of Event Handler

**VB Code Example**

```
Private Sub _controller_CommandReceived(ByVal sender As System.Object, _
          ByVal e As System.EventArgs) Handles _controller.CommandReceived

    Dim command As CommandInfo = _controller.GetCommand()
    If Not command Is Nothing Then
        Console.WriteLine(command.GetCommandText())
    End If
End Sub
```

**C# Code Example**

```
void _controller_CommandReceived(object sender, EventArgs e)
{
    CommandInfo command = _controller.GetCommand();
    if (command != null)
    {
        Console.WriteLine(command.GetCommandText());
    }
}
```

※The goal of the above code is to provide understanding of how to use the code. For an example of how it is actually used, refer to the example in section 4.

## 3.7 Disconnect from the Controller

To disconnect to end communication with the controller, use the “**Close**” method of the controller class.

| **VB Code Example** |
| --- |
| `_controller.Close()` |
| **C# Code Example** |
| `_controller.Close();` |

## 3.8 Check the Command Log

At default, the details of sending and receiving commands between Lightstep and the controller are saved in a file. The log file is saved in the following location.

```
(My Document)¥LightstepLog
```

A separate folder is created for each controller. The folder name starts with the controller type.

| | |
| --- | --- |
| Ethernet type | TCPIP_(IP Address_(Port Number) |
| PCI Bus type | PCI_(Board Number) |
| RS-232C type | COM_(Port Number) |

Also, in each controller folder, there are separate subfolders created by the date.

```
ls_(yyyymmdd)
  yyyy: year
  mm: month
  dd: date
```

Below each folder, one file is saved for approximately every 1Mbytes.

A file name is written as follows.

```
lslog_(Sequence number 3 digits).log
```

If the sequence reaches 999, it returns to 000. Also, when the folder changes because the date has changed, the sequence number becomes 000.

To some extent, you can customize how you want to save the log. For details, refer to the section on practical examples.

# 4. How to Use and Put into Practice

This chapter discusses how Lightstep can be used in a practical way. There are guidelines on using the program, so be sure to read all the information. It is assumed that the user has a basic understanding of Lightstep.

### 4.1 Create an Instance

There are several ways an instance can be created. Choose a method according to how the host application is designed. Normally, Visual Studio and .NET Framework methods are used, see (a) and (b) below. These methods are also introduced in section 3 “How to Use – Basics”.

(a) Dynamically created with code.

(b) Arrange on a form from the Form Designer.

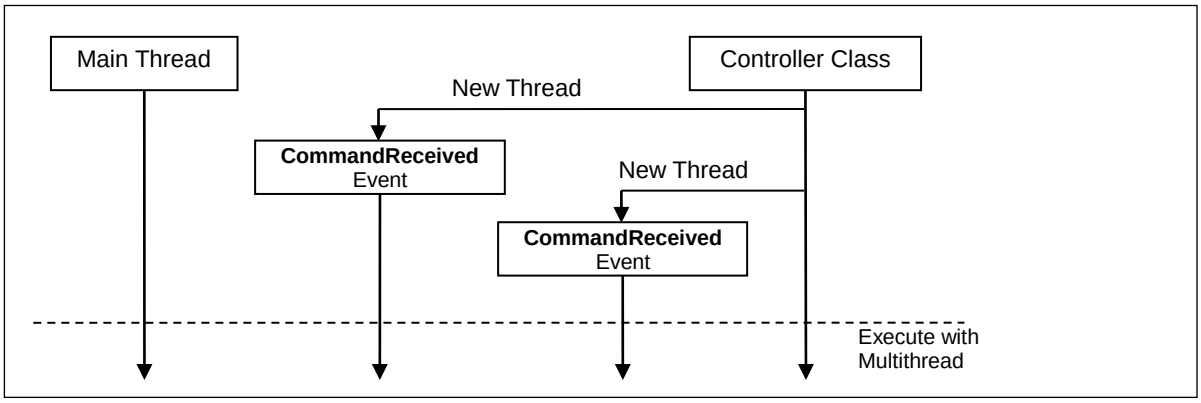
(c) Dynamically created with code and set to synchronize with the form.

Differences in behavior with respect to various methods are explained below.

#### Dynamically Create with Code

When an event from the instance created dynamically with code is raised, it will be executed with a newly created thread of the controller class. The controller class proceeds to the next process[*1]. For this reason, if the host application needs time to process this event, the same type of event can be raised continuously.

(*1) Only the **EndSendCommand** event is executed synchronously from the internal thread of the controller class.



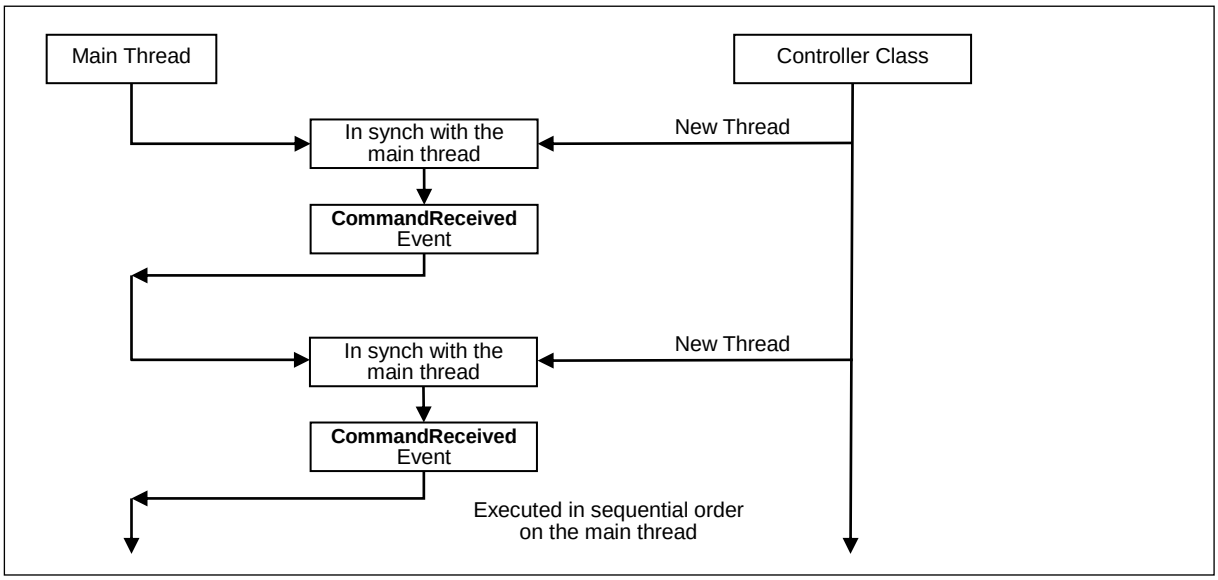

In addition, in order to acess an object on the form, it must be synchronized with the form’s thread. When using an instance created with code, be aware the operation is with the multithread.

How to implement in the event handler is discussed in section “4.5 Commands Received from

the Controller” (p.27).

### Arrange on the Form from the Form Designer

A controller class can be used as a component, and it can be arranged on the form. Events raised by the form-arranged instance will run in synch with the status of the form. For this reason, when the host application is performing the process of a particular event handler, the same event will not be raised (waiting for execution). In addition, an object on the form can be accessed as is.




※Be careful not to destroy the form while communicating with the controller. An exception in the controller class occurs if the form is destroyed. However, since the event cannot be raised, the host application is not notified and the exception is ignored. (In most cases, the host application does not come to a stop.)

※Even when it is set to synchronize with the form, the **EndSendCommand** event of the completed asynchronous transmission will not synchronize with the form. This is caused by “dead lock” between the internal thread of Lightstep and main thread.

### Create Dynamically in Code and Set to Synchronize with the Form

If you want to create the controller class dynamically and synchronize with the form automatically, use the following methods.

**Procedure**

(1) Create an instance.

(2) Set the **SynchronizingObject** property to reference to the form.

**VB Code Example for the Ethernet Type**

```
Public Class Form1
    Private WithEvents _controller As New EthernetController
    Public Sub New()
        _controller.SynchronizingObject = Me
```

**C# Code Example for the Ethernet Type**

```
public partial class Form1 : Form
{
  private EthernetController _controller = new EthernetController();
  public Form1()
  {
    _controller.SynchronizingObject = this;
```

## 4.2 Connect with the Controller

### Specify the Connection Parameter(s)

Parameters are required for connecting with the controller. For example, if you are using an Ethernet controller, you need to specify the IP address and port number.

When connecting, you can execute by specifying the argument(s) of **Connect** method, or execute without specifying the parameter(s) of the **Connect** method.

It is also possible to use the "**Connection String**" parameter specification method. Refer to "**5.1.2 Connection String**" (p.42).

The parameter specifications necessary for different controllers are the following.

| Controller Class | Parameter |
|---|---|
| EthernetController | IP address, port number |
| PciBusController | Board number |
| Rs232cController | COM port number, baud rate, data length, stop bit, parity<br>(Can omit all except the COM port number) |

For more information on implementaiton, refer to section 4.3.4 "Implementation of Synchronous Connection" (p. 16)

**Cancel the Connection**

There is no function to cancel the connection. The Ethernet type controller will time out if it is unable to connect within a certain time (about 20 seconds).

For the Ethernet controller, long connection times can be prevented by sending the PING with timeout to about 3 seconds and check just before connecting.

## 4.3 Communicate by Using a Synchronous Method

Similar to the **BeginSendCommand** method, all the methods starting with "**Begin**" will work for the asynchronous operation. All other methods work for synchronous operation.

### 4.3.1 What is Synchronous Communication

In this section synchronous communication will be explained. A synchronous method runs on the thread that calls the method, and when finshed executing that method, the synchronous method has completed the execution. Only after the host application has finished executing the synchronous method, it can proceed to the next process.

### 4.3.2 List of Synchronous Method

The main synchronous methods of the controller class are the following.

| | |
|---|---|
| Connect | Connect with the controller |
| SendCommand | Send a command |
| GetCommand | Get a command from the controller |
| Close | Disconnect with the controller |

### 4.3.3 Advantages and Disadvantages

Advantages and disadvantages in synchronous communication are the opposite of the advantages and disadvantages of asynchronous communication.

**Advantage**

Can implement in simple coding.

**Disadvantage**

- When communicating with multiple controllers, the process is sequential. For this reason, there is no benefit from using multiple controllers for distributed processing.
- The user interface response may deteriorate. For example, if there are several commands to send and if you press a button on the screen, you will not be able to use the screen until all the commands have been sent.

Synchronous communication is suitable if you use only 1 controller and the host application has no user interface.

### 4.3.4 Implement Synchronous Connection

This section describes how to connect sychronously with the controller.

The **Connect** method is used for synchronous connection. If an error occurs while connecting, the **Connect** method throws an exception. Put the **Connect** method inside the **Try** block and implement the error process in the **Catch** block.
The connection result is determined by the presence or absence of the exception. If an exception occurs, it means that connection was not made. Refer to “4.8 Error Processing” (p. 38) to determine the error type.

**VB Code Example for the Ethernet Type**

```
Try
  _controller.Connect("192.168.1.254", 5003)
Catch err As Exception
  Console.WriteLine(err.Message)
  return
End Try
```

**C# Code for the Ethernet Type**

```
try
{
  _controller.Connect("192.168.1.254", 5003);
}
catch (Exception err)
{
  Console.WriteLine(err.Message);
  return;
}
```

### 4.3.5 Implement Synchronous Transmission

This section discusses the method of sending commands synchronously.

The **SendCommand** method is used. If an error occurs while sending, the **SendCommand** method throws an exception.
Put the **SendCommand** method inside the **Try** block and implement the error process inside the **Catch** block. If an exception occurs, it means that the command was not sent successfully.

If you want to get and use the return command, set up the return value. The return value will be presented in the object of the **Commandinfo** class.

To get the response command, execute the **GetCommandText** method of the **CommandInfo** object.

**VB Code Example for the Ethernet Type**

```
Try
  Dim ack As CommandInfo = _controller.SendCommand("Z")
  Console.WriteLine(ack.GetCommandText())
Catch ex As Exception
  Console.WriteLine(ex.Message)
End Try
```

**C# Code for the Ethernet Type**

```
try
{
  CommandInfo ack = _controller.SendCommand("Z");
  Console.WriteLine(ack.GetCommandText());
}
catch (Exception err)
{
  Console.WriteLine(err.Message);
}
```

#### Command Transmission Timeout

When **Ack** is not returned within a certain time (timeout time) after a command is sent, the **SendCommand** method will throw a **TimeoutException** exception. The timeout time is set up with **CommandTimeout** property of the controller class. The default value is different for each type of controller. The Ethernet type is 10,000 miliseconds (10 seconds).

In addition to the time for packet communication, Lightstep needs time to get commands from the buffer, and if the load on the PC itself is heavy, processing speed slows down. This could cause a timeout.

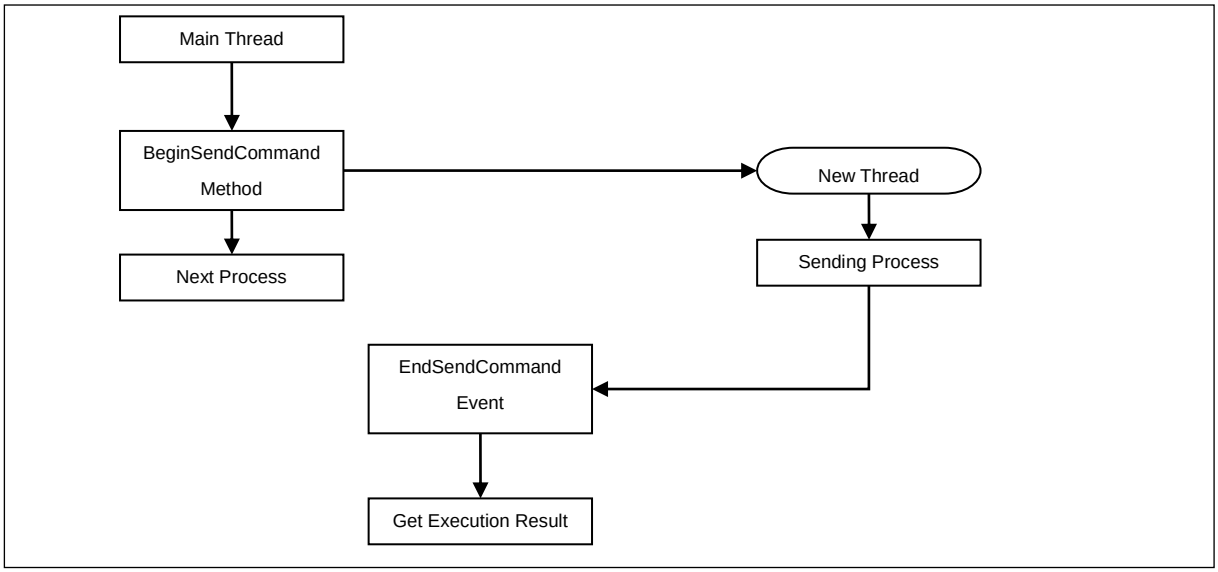
## 4.4 Communicate by Using the Asynchronous Method

The name of the method that starts with "**Begin**", such as **BeginSendCommand,** is a method for the asynchronous operation.

### 4.4.1 Asynchronous Communication

This section explains asynchronous communication with regard to Lightstep. In the asynchronous method, the process is not performed with the thread that calls the asynchronous method. It is performed with a new thread. For this reason, the thread that calls the asynchronous method can gain control immediately and instantly move to the next process.

The execution result of the method being called is obtained with an event handler. When a process is completed with a different thread, a completion event corresponding to the asynchronous method will occur. Event names start with "**End**" as in **EndSendCommand**.




Execution of the Asynchronous Transmission

### 4.4.2 List of the Asynchronous Method

A list of the asynchronous method is shown below.

| | |
|---|---|
| BeginConnect | Connect with the controller asynchronously |
| BeginSendCommand | Send command asynchronously |
| BeginClose | Disconnect from the controller asynchronously |

The following are the execution completion events.

| | |
|---|---|
| EndConnect | Occurs at the completion of the asynchronous connection. |
| EndSendCommand | Occurs at the completion of sending a command asynchronously. |
| EndClose | Occurs at the completion of the asynchronous disconnection. |

### 4.4.3 Advantages and Disadvantages

Advantages and disadvantages of asynchronous communication are the opposite of the advantages and disadvantages of synchronous communication.

**Advantages**

- When communicating with multiple controllers, a command can be sent to each controller almost at the same time, and it is possible to have a highly responsive light module control.
- The control PC's performance is enhanced because the controller's communication processing is performed not by the main thread but by a different thread and can perform in parallel with non-communication processing.
- You can create applications that will not compromise the responsiveness of the user interface despite frequent communication with the controller.

**Disadvantage**

- Programming will become complex compared to the synchronous communication.

It is advantageous to use asynchronous communication when multiple controllers are used.

### 4.4.4 Implement Asynchronous Connection

An explanation on how to connect asynchronously with the controller is given below.

Use the **BeginConnect** method. The parameter setting of the asynchronous connection is the same as that of the synchronous connection. If an error occurs while the connection is being made, the **BeginConnect** method will throw an exception. Put the **BeginConnect** method inside the **Try** block and implement the error process inside the **Catch** block.

The exception here means an error occurred before the connection process. As a result, the asynchronous connection process did not start.

**VB Code Example for the Ethernet Type**

```
Try
  _controller.BeginConnect("192.168.1.254", 5003)
Catch err As Exception
  Console.WriteLine(err.Message)
End Try
```

**C# Code Example for the Ethernet Type**

```
try
{
  _controller.BeginConnect("192.168.1.254", 5003);
}
catch (Exception err)
{
  Console.WriteLine(err.Message);
}
```

To confirm the connection result, the **EndConnect** event is implemented. If the connection could not be made, the **Exception** object is set in the **Error** property of the **EndConnectEventArgs** object passed by the parameters of the event handler. This exception occurs during the connection processing.

If the **Error** property is empty (**Nothing** for VB, **null** for C#), it means that the connection was finished normally.

**VB Code Example**

```
Private Sub _controller_EndConnect(ByVal sender As System.Object, _
    ByVal e As EndConnectEventArgs) Handles _controller.EndConnect
  If Not e.Error Is Nothing Then
    console.WriteLine(e.Error.Message);
    Return
  End If
    'Processing after connecting
```

**C# Code Example**

```
void _controller_EndConnect(object sender, EndConnectEventArgs e)
{
  if (e.Error != null)
  {
    Console.WriteLine(e.Error.Message);
    return;
  }
  // Processing after connecting
```

### Reason for Using the Asynchronous Connection Method

In asynchronous connection, the connection result is obtained with the event handler. Because the post connection process is coded here, the readability of the program may drop.

However, because the connection processing is executed with the sub-thread, the host application will not experience a non response status.

It takes about 20 seconds for the Ethernet controller to time out before a connection is made. For this reason, if the controller does not get power and at the same time tries to connect, the program will enter into the freeze state.

### 4.4.5 Implement Asynchronous Transmission

This section explains the method of sending commands asynchronously.

The **BeginSendCommand** method is used. If an error occurs during the start of the asynchronous transmission process, the **BeginSendCommand** throws an exception. Put the **BeginSendCommand** method inside the **Try** block and implement the error process inside the **Catch** block.

The exception here means an error occurred before the transmission process. As a result, the asynchronous transmission process did not start.

**VB Code Example**

```
Try
  _controller.BeginSendCommand("Z")
Catch ex As Exception
  Console.WriteLine(ex.Message)
End Try
```

**C# Code Example**

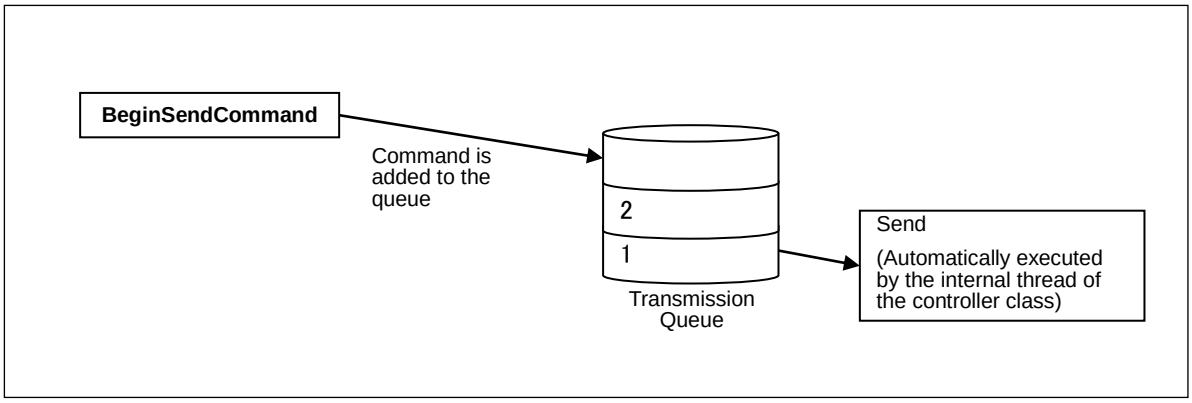
```
try
{
  _controller.BeginSendCommand("Z");
}
catch (Exception err)
{
  Console.WriteLine(err.Message);
}
```

#### Asynchronous Transmission Structure

When the **BeginSendCommand** method is executed, the command is not sent by another thread, but stored in the transmission queue (buffer queue).

At this moment, if transmission process is already running, it continues to send commands until there are no remaining commands in the transmission queue. However, if transmission process is not running, the transmission process will start with a new thread.

Thus, in the asynchronous transmission, with the instance of the controller class, the communication procedure of “sending command → responding command received → and sending the next command” is maintained.




Asynchronous Transmission

### Check the Transmission Result

The **EndSendCommand** event is used to get the command response and check success or failure of transmission.

If an error occurs, an **Exception** object will be set in the **Error** property of the **EndSendCommandEventArgs** object passed with the parameter of the event handler. This exception occurs during the transmission processing.

If the **Error** property is empty (**Nothing** for VB, **null** for C#), it means that the transmission was finished normally.

### Process Options after Error Has Occured

The **EndSendCommand** event will occur synchronously from the controller class's command transmission thread. The controller class continues with the next processing after waiting for the completion of the event handler. For this reason, the host application has a chance to choose the next action when a command transmission error occurs.

Before exiting the event handler, with the **NextActions** enumerator, specify the next process in the **ErrorNextAction** property of the **EndSendCommandEventArgs** object which was passed with the parameter of the event handler. The following are the processing types.

| Type of Processing | Meaning |
|---|---|
| NextActions.Ignore | No process is performed for an error caused by the command transmission. The command that caused an error is removed from the asynchronous transmission queue. If there is another command, the next command is sent. By default this value is set. |
| NextActions.Retry | Resend the command that resulted in an error.<br>Before the retry, it is recommended that you first disconnect the controller. After you are able to reconnect, you specify this value and finish the event handler. (※The transmission queue will be retained even when the **Close/Connect** method is performed.) |
| NextActions.Stop | No process is performed for an error caused by the command transmission. Also, the asynchronous transmission queue will be cleared (all standby commands are canceled) and the transmission thread will end.<br>Specify this value to immediately stop the light module control. |

If the **EndSendCommand** event handler is not implemented and an error occurs, the **NextActions.Ignore** will be set up.

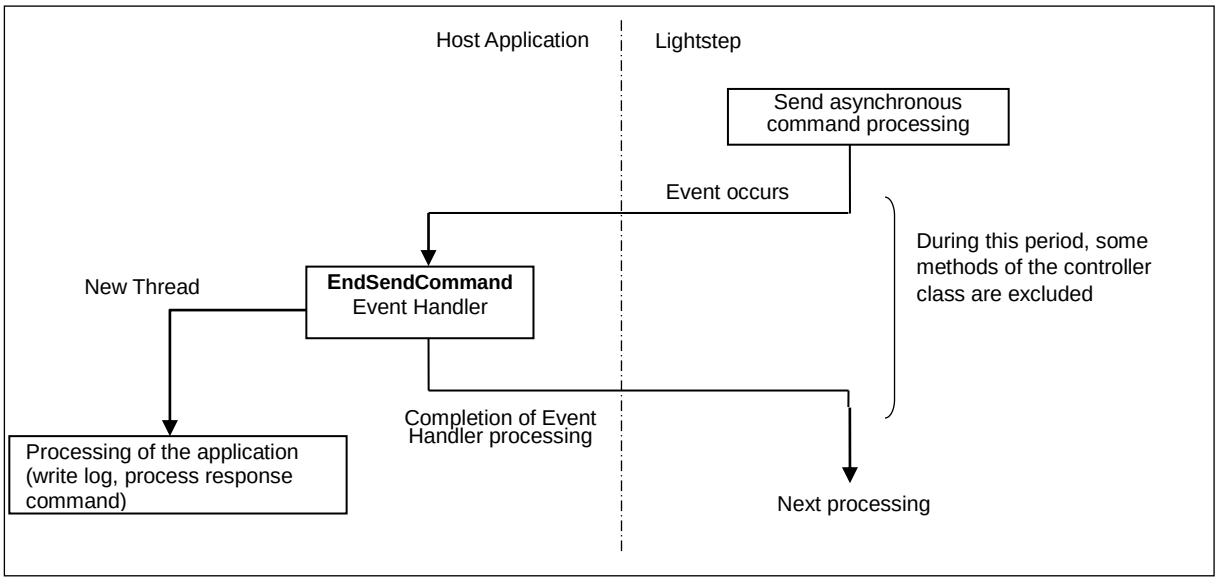
### What to Watch for When Implementing the "EndSendCommand" Event Handler

Because the **EndSendCommand** event occurs synchronously by Lightstep's command transmission thread, the next command cannot be sent until the event handler has finished the process. In addition, because during the transmission process other methods are excluded from being processed, a deadlock among threads can occur.

For example, when the **SendCommand** method is used in the **EndSendCommand** event, **TimeoutException** is raised. During the synchronous command transmission, the asynchronous command transmission waits for the transmission queue to become empty. But when it is called from the **EndSendCommand** event handler, the asynchronous transmission can't complete, and the **SendCommand** method will timeout.

Determining errors and specifying the next action should be performed in the **EndSendCommand** event handler, and all other processing should be performed by creating a new thread.

※You can use the **Close** method and **Connect** method for the resending process. For more information on resending, refer to section "4.8.5 Resend the Transmission Command" (p. 40).




### Get Response Command

To get the response command, the **EndSendCommandEventArgs** object is used for passing the parameter of the event handler. The response command can be obtained from the **GetCommandTex** method of the **CommandResponded** property (**CommandInfo** type object) of the **EndSendCommandEventArgs** object. (You can also get byte array type by using the **GetCommandBytes** method.)

Also, to get the transmission command, use the **CommandSent** property of the **EndSendCommandEventArgs** object.

With the **ToString** method of the **CommandInfo** class, you can also get the same result as you get with the **GetCommandText** method.

### Example of Implementing the "EndSendCommand" Event Handler

In this example, when a transmission error occurs, **NextActions.Stop** is specified to the **ErrorNextAction** property to clear the asynchronous transmission queue and, with a new thread, a method is called to stop the light mondule control.

**VB Code Example**

```
Private Sub _controller_EndSendCommand(ByVal sender As System.Object, _
    ByVal e As EndSendCommandEventArgs) Handles _controller.EndSendCommand
  If e.Error Is Nothing Then
    Console.WriteLine("Source Command = {0}, Ack = {1}", _
      e.CommandSent.ToString(), _
      e.CommandResponded.ToString())
  Else
    Console.WriteLine(e.Error.Message)
    e.ErrorNextAction = NextActions.Stop
      ' Example of calling the stop process with a new thread.
    ThreadPool.QueueUserWorkItem(New WaitCallback(AddressOf StopWork))
  End If
End Sub
```

**C# Code Example**

```
void _controller_EndSendCommand(object sender, EndSendCommandEventArgs e)
{
  if (e.Error == null)
  {
    Console.WriteLine("Source Command = {0}, Ack = {1}",
      e.CommandSent.ToString(),
      e.CommandResponded.ToString());
  }
  else
  {
    Console.WriteLine(e.Error.Message);
      e.ErrorNextAction = NextActions.Stop;
      //Example of calling the stop process with a new thread.
      ThreadPool.QueueUserWorkItem(new WaitCallback(StopWork));
    }
  }
}
```

### 4.4.6 Implementation of Asynchronous Disconnection

This section explains how to perform asynchronous disconnection from the controller.

The **BeginClose** method is used. If an error occurs when starting the disconnection, the **BeginClose** method will throw an exception. Put the **BeginClose** method inside the **Try** block, and implement the error process inside the **Catch** block.

The exception here means an error occurred before the disconnection process. The result is the asynchronous disconnection process did not start.

**VB Code Example**

```
Try
  _controller.BeginClose()
Catch err As Exception
  Console.WriteLine(err.Message)
End Try
```

**C# Code Example**

```
try
{
  _controller.BeginClose();
}
catch (Exception err)
{
  Console.WriteLine(err.Message);
}
```

With the **EndClose** event, the confirmation of the disconnection result is implemented. If an error occurs during the disconnection, the **Exception** object will be set in the **Error** property of the **EndCloseEventArgs** object passed by the parameter of the event handler. This exception occurs during the disconnection processing.

If the **Error** property is empty (**Nothing** for VB, **null** for C#), it means that the disconnection was finished normally.

**VB Code Example**

```
Private Sub _controller_EndClose(ByVal sender As Object, _
    ByVal e As EndCloseEventArgs) Handles _controller.EndClose
  If Not e.Error Is Nothing Then
    Console.WriteLine(e.Error.ToString())
  End If
End Sub
```

**C# Code Example**

```
void _controller_EndClose(object sender, EndCloseEventArgs e)
{
  if (e.Error != null)
  {
    Console.WriteLine(e.Error.ToString());
  }
}
```

#### Reason for Using the Asynchronous Method to Disconnect

If you send a command with the **BeginSendCommand** method and execute the **Close** method immediately after, the last command you wanted to send will remain unsent despite **Close** was executed. If, however, the **BeginClose** is used, the disconnection process takes place after all the commands have been sent with asynchronous transmission.

### 4.4.7 Using the Synchronous Method at the Same Time

You can combine both the synchronous and asynchronous methods. For example, connecting synchronously and sending a command asynchronously will not cause any problems. However, it is important to understand how they work when you are combining synchronous transmission with asynchronous transmission. Unless you have a good reason to combine them, it is better to use a single method.

#### Processing Order of Synchronous Transmission and Asynchronous Transmission

If you perform asynchronous transmission after synchronous transmission, this simply means after a command is sent synchronously, the asynchronous command is sent.

Conversely, if synchronous transmission is executed after asynchronous transmission, "**Standby**" will occur until the asynchronous queue becomes empty. When the queue becomes empty, that is, after all the commands are processed with asynchronous transmission, the synchronous transmission is executed.

"**Standby**" is automatically performed within the **SendCommand** method of the controller class. Therefore, the host application does not have to be aware of this. However, "**Standby**" has a timeout setting. Unless asynchronous transmission is completed within the timeframe set with the **SynchronousTimeout** property of the controller class, the **SendCommand** method will throw an exception, **TimeoutException**.

The default value of the **SynchronousTimeout** property is 10,000 miliseconds.

If asynchronous transmission and synchronous transmission are combined and used, the order may not turn out the way you intended, so please be aware of this.

#### Wait for the Completion of Asynchronous Transmission

In order to wait until all the commands stored in the asynchronous transmission queue are sent, the **WaitSendTask** method is used. It waits only for the time specified with the parameter, and if the queue is not emptied within the timeframe, the return value, "**false**", is returned. (An exception is not raised.)

## 4.5 Commands Received from the Controller

A command is sent from the controller when the light module's button is pressed or a barcode is scanned with a barcode reader connected to the barcode reader interface. With Lightstep, this command is received internally, saved sequentially in the internal buffer, and reports to the host application with an event.

The host application will get and use the commands that Lightstep has saved. Below is an explanation of the procedure for getting a command in the event handler and using it.

### 4.5.1 General Flow of Command Receiving Process

(1) The **CommandReceived** event handler's definition is assigned to the instance of the controller class.

(2) Get the received command with the **GetCommand** method of the controller object in the event handler.

(3) Because the command received may be linked with multiple addresses, split it into individual address and process.

### 4.5.2 Implementaiton Cases

This section provides a step-by-step explanation on the implementation of the event handler.

Implementation Example – The Entire Event Handler

**VB**

```
Private ReadOnly _lockObject As New Object
Private Sub _controller_CommandReceived(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles _controller.CommandReceived
  SyncLock _lockObject
    Dim command As CommandInfo = _controller.GetCommand()
    If command Is Nothing Then
      Return
    End If
    'Split the address information included in the command as the "AddressInfo" object
    Dim list As AddressInfo() =  _
      AddressInfo.SplitCommand(command.GetCommandBytes())
    For Each address As AddressInfo In list
      Console.WriteLine("Address = {0}, Status = {1}", _
        address.Address, _
        address.Status)
      'Process the address that had action.
      'HandleModule(address)
    Next
  End SyncLock
End Sub
```

**C#**

```
private readonly object _lockObject = new object();
void _controller_CommandReceived(object sender, EventArgs e)
{
  lock (_lockObject)
  {
    CommandInfo command = _controller.GetCommand();
    if (command == null)
      return;
    // Split the address information included in the command as the "AddressInfo" object.
    AddressInfo[] list = AddressInfo.SplitCommand(command.GetCommandBytes());
    foreach (AddressInfo address in list)
    {
      Console.WriteLine("Address = {0}, Status = {1}",
        address.Address,
        address.Status);
      // Process the address that had action.
      //HandleModule(address);
    }
  }
}
```

The following explains the main points in sequence.

**VB**

```
Private ReadOnly _lockObject As New Object
Private Sub _controller_CommandReceived(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles _controller.CommandReceived
  SyncLock _lockObject
```

**C#**

```
private readonly object _lockObject = new object();
void _controller_CommandReceived(object sender, EventArgs e)
{
  lock (_lockObject)
  {
```

Try to synchronize the process inside the event handler. In this example, VB's "**SyncLock**" and C#'s "**lock"** statement are used. If a new event occurs while the event is being processed, the new event must wait.

Here, a single exclusive object is specified, but if event handlers of multiple controllers are combined, use an exclusive object for each controller so that responses from multiple controllers can be processed in parallel.

**VB**

```
Dim command As CommandInfo = _controller.GetCommand()
If command Is Nothing Then
  Retrun
End If
```

**C#**

```
CommandInfo command = _controller.GetCommand();
if (command == null)
  return;
```

With the **GetCommand** method, get the command received. The command received will appear as the **CommandInfo** type. If there is no command received, **null** is returned.

**VB**

```
Dim list As AddressInfo() = _
  AddressInfo.SplitCommand(command.GetCommandBytes())
```

**C#**

```
AddressInfo[] list = AddressInfo.SplitCommand(command.GetCommandBytes());
```

Get the command text with the **GetCommandBytes** method of the **CommandInfo** object in byte array(*1) and break down into individual address information with the **SplitCommand** method, which is a static method of the **AddressInfo** class. The **SplitCommand** method is a way to break down the individual address information within the "**t**" command.

(*1)You can get the command with the **GetCommandText** method as a character array, but if binary data is included in the input data of the barcode reader interface (BCR I/F), characters may not show up.

**VB**

```
For Each address As AddressInfo In list
  Console.WriteLine("Address = {0}, Status = {1}", _
    address.Address, _
    address.Status)
  'Processes the address that had an action.
  'HandleModule(address)
Next
```

**C#**

```
foreach (AddressInfo address in list)
{
  Console.WriteLine("Address = {0}, Status = {1}",
    address.Address,
    address.Status);
  // Processes the address that had an action.
  //HandleModule(address);
}
```

The address information that has been split can be obtained as the **AddressInfo** object array. The address and status can be obtained from the property of the **AddressInfo** class object. (You do not necessarily have to use the **AddressInfo** class to cut out the address. In certain situations, you can make your own cutout.)

These values (or reference to the **AddressInfo** object) are passed to the host application processing when light module buttons are pressed. (For details on the **AddressInfo** object, refer to section 4.7.3 Analysis and Breakdown of the "**t**" Command, p.35)

## 4.6 Use of the Command Log

By default, the contents of the commands sent to and received from the controller are saved in the log file with Lightstep.

### 4.6.1 Overview of the Operation

- Below each folder, a controller folder is created for each controller. And below each controller folder, a dated folder is created by the current date. Then the log file is created in the dated folder.
- When the log file reaches a certain volumn, a new log file is created. Each newly created log file will get a name followed by a 3-digit sequence number.
- For each transmission frame, a line break (CRLF) is added at the end of the line.
- The log file is recorded as a binary file.
- The dated folder after a certain period will be deleted.
- If a stop file for log recording is created in each controller folder, the log can be externally stopped.
- With the PCI bus and RS-232C types of controllers, there is a simple log mode to control the log size.

### 4.6.2 Location of the Saved Log File

As a basic folder, log files are created in the **LightstepLog** folder in the user's **My Documents**.

Below the basic folder, separate folders are created for each controller type and parameter.

| | |
|---|---|
| Ethernet type | TCPIP_(IP address)_(Port number) |
| PCI bus type | PCI_(Board number) |
| RS-232C type | COM_(Port number) |

Below each controller folder, dated folders are created by date. If the date crosses over to the next day while communication is taking place, a new folder is created and a new file is created inside that folder.

ls_(yyyymmdd)
yyyy: year
mm: month
dd: date

A log file is saved in the dated folder. The file name is as follows:

| |
|---|
| lslog_(any text )(3-digit sequence number).log |

If the sequence reaches 999, it returns to 000.

When the file reaches a certain capacity, a new file will be created with the next sequence number, and a log is recorded. By default, the file size is 1Mbyte.

If log files already exist at the the start of communication, log is appended to the file with the last sequence number.

There is no default for “any text”. It can be set with the host application according to your need.

Log Directory Hierarchy (the yellow portion is folder.)

```
(My Documents)
  ├─LightstepLog
  │  │
  │  ├─TCPIP_192.168.1.253_5003
  │  │  └─ls_20100519
  │  │        lslog_000.log
  │  ├─TCPIP_192.168.1.254_5003
  │  │  ├─ls_20101007
  │  │  │    lslog_000.log
  │  │  └─ls_20101013
  │  │       lslog_000.log
  │  │       lslog_001.log
  │  ├─PCI_1
  │  │  └─ls_20101008
  │  │        lslog_000.log
  │  ├─COM_1
  │  │  └─ls_20101014
  │  │        lslog_000.log
```

### 4.6.3 How to Read the Command

Logs are created as binary files, and most of them are expressed as ASCII characters. For this reason, you can see logs in the text editor, such as Notepad. Also, if you open a binary editor, you can check the binary codes.

Each line is configured with the following content.

| |
|---|
| (Log Time)(Space)(Log Type)(Space)(Communication Data)(CRLF) |

| | |
|---|---|
| Log Time | Time the log was created. |
| Log Type | i> Information message (Output when session starts and ends)<br>!> Warning message (ver. 1.0 cannot output the warning message)<br>S> Command sent from the application to the controller<br>R> Command received from the controller |
| Communication Data | Sent or received data itself. The data that cannot be expressed in text will have missing characters or will not show, but can be checked, for example with binary editor.<br>If the log type is “Information”, “Warning”, there will be a message. |

The 1st line of the log file will record the version number of Lightstep and the date it was created.

※Because the first line was recorded the moment the data was written, it may be later than the time of the 2nd line.

**Example of Log Output of the Ethernet Controller Type**

```
15:05:49.452 i> Lightstep version 1.0.0.0 / Log created at 2011/07/21.
15:05:49.450 i> Session started.
15:05:50.067 S> 0010001Z————————————————————
15:05:50.072 R> 0010001o————————————————————
15:05:50.078 S> 0020013P100000112345————————————————————
15:05:50.080 R> 0020001o————————————————————
15:05:52.127 R> 0010007t000100————————————————————
15:05:52.130 S> 0010001O————————————————————
15:05:54.034 S> 0030001Z————————————————————
15:05:54.041 R> 0030001o————————————————————
15:05:54.045 i> Session closed.
```

### 4.6.4 Automatic Deletion of Log

The log folders dated before the specified date will be deleted automatically.

| **Retention Period** | **14 Days (Default)** |
|---|---|
| Delete timing | When implementing the controller connection method (Connect/BeginConnect). Executed regardless of whether the connection was successful or failure. |

If the log deletion fails when the connection method (**Connect/BeginConnect**) is executed, it throws an exception called **LoggingException**, and the connection fails.

### 4.6.5 Change the Log Setting

To change the log setting, use the **Logger** property of the controller class. Change the values of the properties of the specified **LogFileSettings** object.

| **Property Name** | **Explanation** | **Default Value** |
|---|---|---|
| Enabled | Log enabled with “True”<br>Log disabled with “False” | True |
| Expiration | Number of days the log is retained | 14 |
| LogDirectory | Directory where the log is stored | **My Documents** folder |
| LogName | Character array setup in the log file’s “any text” | (None) |
| LogSize | Maximum size (bytes) per file | 1,048,576 |

The log setup is used when you are connecting to the controller.　For the setup, use the timing before the connection.

### 4.6.6 Stop the Log from Outside the Application

The log can be stopped even after the application has been implemented to get the log. From Windows Explorer, create an empty file called “**stop**” in the log folder.

Folder to place the “**stop**” file.

| |
|---|
| (My Documents or specified location)¥LightstepLog¥(folder classified by controller)¥ |

If multiple controllers are used, folders are different for different controllers.

## 4.7 Processing Commands

Lightstep has functions to create and analyze commands.

- Binary conversion
- Help create repeatable commands
- Break down the “**t**” command

### 4.7.1 Binary Conversion

In the command, there are values that cannot be expressed in characters.　In this case, have values specified in hexadecimal, and when it is time to send, Lightstep can convert them into binary values and send.

In default, this operation is valid.

#### How to Use

(1) Specify **True** to the **BinaryMode** property of the controller class. (By setting it just once after creating an instance, the setup will be retained.)

(2) When specifying a binary value in the command character array, describe it with “**$xx**” (xx in hexadecimal value).

**VB Code Example**

```
_controller.BinaryMode = True
_controller.SendCommand("Am10001$33$11$23")
```

**C# Code Example**

```
_controller.BinaryMode = true;
_controller.SendCommand("Am10001$33$11$23");
```

In the above example, ”$33” is converted to 1 byte of “33h”, ”$11” to 1 byte of 11h, and ”$23” to 1 byte of 23h, and then sent.

When the **BinaryMode** is **False** and the above commands are sent, the character array will be sent as they are. (The “**n**” response will be returned).

### 4.7.2 Create Repeatable Commands

There are commands for specifying multiple addresses. They should be used because these commands will reduce the number of transmission commands. However, due to the limitation on the command length, the command must be divided into multiple commands when there are many addresses.

To process multiple addresses, Lightstep provides a way to shorten the command and divide it into multiple commands through the **RepeatableCommand** class.

**How to Use**

(1) Create an instance of the **RepeatableCommand** class.

(2) Keep adding the character string of the reapeatable portion with the **Add** method of the **RepeatableCommand** object.

(3) With the **CreateCommand** method of the **RepeatableCommand** object, specify the first part of the command and the splitting method and execute. First part of the command means from the command header until repeating begins. The splitting method is specified with a value listed in the **SplitType** enumerator.

(4) Types of Splitting Method

| | |
|---|---|
| SplitType.ByteSize | Command splitting is determined by the command’s data length (number of bytes). If the “maximum command length” of the command reference is in bytes, specify this. |
| SplitType.RepeatCount | Command splitting is determined by the number of repeats. If the number of addresses represents the “Maximum Command Length” of the command reference, specify this. |

When this method is executed, the command is created according to the specified splitting method. The return value is shown in string type array.

(5) The commands obtained with (3) are all sent.

**VB Code**

```
Dim repeater As New RepeatableCommand
For i As Integer = 1 To 500
  repeater.Add(i.ToString("0000") & "12345")
Next
Dim results As String() = _
  repeater.CreateCommand("P100", RepeatableCommand.SplitType.ByteSize)
For Each command As String In results
  _controller.SendCommand(command)
Next
```

**C# Code**

```
RepeatableCommand repeater = new RepeatableCommand();
for (int i = 1; i <= 500; i++)
{
  repeater.Add(i.ToString("0000") + "12345");
}
string[] results =
  repeater.CreateCommand("P100", RepeatableCommand.SplitType.ByteSize);
foreach (string command in results)
{
  _controller.SendCommand(command);
}
```

### 4.7.3 Analysis and Breakdown of the “t” Command

The “**t**” command is a command that the controller sends when the light module’s button is pressed or when an input is made from the barcode reader interface. When the “**t**” command is used to report that the light module button has been pressed, multiple addresses may be connected.

Also, when quantity revision is made with the light module, the revised quantity will also be included in the “**t**” command. **Lightstep** analyzes information from the “**t**” command and provides a function to create an object for each address with the **AddressInfo** class.

With the PCI bus type controller and RS-232C type controller, the “**h**” response command is used for the “**H**” command, and these commands can also be used.

#### How to Use

(1) Specify the the “**t**” command (or “**h**” command) to an argument and execute the **SplitCommand** method, a static method of the **AddressInfo** class.

(2) An array of the **AddressInfo** class can be obtained with the return value. Items of the array will be created only for the number of addresses included in the “**t**” command.

(3) Use the **AddressInfo** object and execute the processing of each address. Addresses and completion status are stored in the **AddressInfo** objects. The input data from the barcode reader interface can be obtained from the **InputData** property. When binary data is included in the input data, the **GetInputDataBytes** method is used to get the data in byte array.

**VB Code Example**

```
Dim command As CommandInfo = _controller.GetCommand()
If command Is Nothing Then
  Return
End If
'Split the address information included in the command as the "AddressInfo" object
Dim list As AddressInfo() = _
  AddressInfo.SplitCommand(command.GetCommandBytes())
For Each address As AddressInfo In list
  Console.WriteLine("Address = {0}, Status = {1}", _
    address.Address, _
    address.Status)
Next
```

**C# Code Example**

```
CommandInfo command = _controller.GetCommand();
if (command == null)
  return;
'Split the address information included in the command as the "AddressInfo" object
AddressInfo[] list = AddressInfo.SplitCommand(command.GetCommandBytes());
foreach (AddressInfo address in list)
{
  Console.WriteLine("Address = {0}, Status = {1}",
    address.Address,
    address.Status);
}
```

### Various Forms of the "t" Command

There are various forms of the "**t**" command pattern. A pattern that can be cut out with the **AddressInfo** is shown below. To process other forms of the "t" command, you need to create a process to break down the data.

■Get the Revised Quantity Data

The format supports the light modules quantity revision. The revised quantity can be obtained with the **InputData** property. The value stored in the **InputData** property is the data enclosed in "<" and ">".

| **Example** |
| --- |
| When the command received is "t000100<00003>", the value of the **InputData** property is "00003". |

■Get the Quantity Revision Data of Each Page with the Page Turning Controller

The page-turning controller supports the quantity revision of each page and the "**Fn**" operation.

The data of each page can be obtained with the **Pages** property of the **AddressInfo** object. The **Pages** property is represented by the **Dictionary** generic class, and indexer for the key and the data are stored for each page.　Because only the essential pages are stored, the values can be extracted using a loop structure.

**VB Code Example**

```
Dim command As String = "t000100<0-00002><3-00000>"
Dim list As AddressInfo() = AddressInfo.SplitCommand(command)
For Each pair As KeyValuePair(Of Integer, String) In list(0).Pages
  Console.WriteLine("index = {0}, data = {1}", pair.Key, pair.Value)
Next
```

**C# Code Example**

```
string command = "t000100<0-00002><3-00000>";
AddressInfo[] list = AddressInfo.SplitCommand(command);
foreach (KeyValuePair<int, string> pair in list[0].Pages)
{
  Console.WriteLine("index = {0}, data = {1}", pair.Key, pair.Value);
}
```

Output Result

```
index = 0, data = 00002
index = 3, data = 00000
```

■Response Data of the 6-digit Light Module

Quantity revision data of the 6-digit light module is supported as well as the quantity revision data of page-turning.　The usage method is the same as the method discussed earlier.

## 4.8 Error Processing

Use an exception mechanism of "**.NET Framework**" to process Lightstep errors. An error from processing the internal thread of Lightstep is passed as an exception object by the **ErrorRaised** event. An error during the process of an asynchronous method starting with "**Begin**" is passed by an exception object raised by the event starting with "**End**".

Errors can occur from hardware damage or communication failure. It can occur when you try to send a command to the controller that is disconnected or from programming mistakes, such as a wrong command format.

### 4.8.1 Errors when Executing Methods

Included here are possible exceptions for all methods. Therefore, execution of error processing should be performed by using the "**Try~Catch**" syntax of the individual client application within an appropriate range. It is not always necessary to peform on a per-method basis.

**VB Code Example**

```
Try
  _controller.SendCommand("Z")
  _controller.SendCommand("X8101     123-45")
  _controller.SendCommand("P1000001    50002    2")
Catch ex As Exception
  Console.WriteLine(ex.ToString())
End Try
```

**C# Code Example**

```
try
{
  _controller.SendCommand("Z");
  _controller.SendCommand("X8101     123-45");
  _controller.SendCommand("P1000001    50002    2");
}
catch (Exception err)
{
  Console.WriteLine(err.ToString());
}
```

### 4.8.2 Errors from the Asynchronous Method

The method that starts with "**Begin**" is paired with the "**End**" event. (Example: The **BeginConnect** method is paired with the **EndConnect** event. You can find out the result of the asynchronous execution result from the completion event.

**VB Code Example**

```
Private Sub _controller_EndConnect(ByVal sender As System.Object, _
    ByVal e As EndConnectEventArgs) Handles _controller.EndConnect
  If Not e.Error Is Nothing Then
    console.WriteLine(e.Error.ToString());
    Return
  End If
```

**C# Code Example**

```
void _controller_EndConnect(object sender, EndConnectEventArgs e)
{
  if (e.Error != null)
  {
    Console.WriteLine(e.Error.ToString());
    return;
  }
```

※ Exception can also occur for a method that starts with "**Begin**".

### 4.8.3 Errors Caused by the Internal Thread

If an error occurs while a thread from Lightstep is being processed, an **ErrorRaised** event will occur. You can obtain the exception with the **Erro**r property of the **ErrorRaisedEventArgs** object passed by this event.

※ An error in the asynchronous method starting with "**Begin**" will not be reported with the **ErrorRaised** event. The error is obtained in the event name that starts with "**End**"

**VB Code Example**

```
Private Sub _controller_ErrorRaised(ByVal sender As System.Object, _
  ByVal e As ErrorRaisedEventArgs) Handles _controller.ErrorRaised
  Console.WriteLine(e.Error.ToString())
End Sub
```

**C# Code Example**

```
private void _controller_ErrorRaised(object sender, ErrorRaisedEventArgs e)
{
  Console.WriteLine(e.Error.ToString());
}
```

### 4.8.4 Get Additional Error Information

On occasion, information that has been added to the exception object that has been obtained with the **ErrorRaised** event or from processing exceptions will be included. It is stored in the **Data** property of the **ErrorRaisedEventArgs.Error** object. The key name is **string** type. The value will differ depending on the key value. The following is the saved value.

| Key Name | Value Type | Explanation |
|---|---|---|
| DataBytes | Byte type array | Shows the data related to an error |

※It is very rare for data to be stored in the **Data** property of the exception object. The assumption is that you use it for investigating a special obstacle.
Currently, with the Ethernet type of controller, only when a damaged frame is received and passed to the **ApplicationException** is the damaged frame stored in that **Data** property.

### 4.8.5 Resend the Transmission Command

If the communication path between the controller and light module control PC (PC that operates the host application) is in a very unstable environment, you may want to maintain the communication by resending a transmission command. Introduced below is an example of how Lightstep is used to implement the resending process in the application.

**When Performing Synchronous Transmission (SendCommand Method)**

(1) When the **TimeoutException** exception is captured with the **SendCommand** method, the following resending process starts.

(2) Execute the **Close** method once, then do the **Connect** method in order, and try to reconnect with the controller.

(3) Next resend processing should be done only when the connection (2) was successful. (If necessary, try to reconnect a few times.)

(4) Resend the command. If the Ethernet type of controller is used, use the **ResendCommand** method. This method sends the very last command with the same sequence number. When resending is successful, the normal positive response (e.g., "**o**") or the "**n1**" command is returned. The "**n1**" means the command sent in the beginning has been received and the resending portion was discarded.

If **TimeoutException** exception was raised, resending has failed (no response). If necessary, retry several times starting from (2) to reconnect.

※ If the **SendCommand** method is used instead of the **ResendCommand** method, the sequence number (rejected by the command frame) of the command frame will be changed. Then the controller is unable to determine what to do with the resent command, and the command may be executed in duplicate.

※ If the **ResendCommand** is used when the **SendCommand** has never been executed, the applicaiton will throw a **NullReferenceException** exception.

The **ResendCommand** method can be used for the PCI bus or RS-232C types of controllers, but they send only the last transmission method. In terms of function, it is not different from the **SendCommand** method.

**When Performing Asynchronous Transmission (BeginSendCommand Method)**

(1) If the **Error** property of the **EndSendEventArgs** object passed within the **EndSendCommand** event handler is the object of the **TimeoutException** class, the following resend process starts.

(2) Try to reconnect the controller by first performing the **Close** method followed by the **Connect** method. At this moment, if there is a possibility for the **BeginSendCommand** being executed with a different thread, do the thread exclusion to prevent **BeginSendCommand** from being executed when the controller is reconnected.

(3) Next transmission processing will proceed only when connection was successful at step (2). (If necessary, try reconnecting a few times.)

(4) Set up **NextActions.Retry** to **ErrorNextAction** property of the **EndSendEventArgs** object. When exiting the event handler, resend the very last command you sent. The transmission result is sent by the **EndSendCommand** event, so repeat the process from step (1). To prevent sending an unlimited number of times, enter the resending count and resending completion check to limit the number of resends.

When resending is successful, the **Error** property of the **EndSendEventArgs** object will be empty. (**Nothing** for VB, **null** for C#). ACK will be returned with "**o**" or "**n1**".

# 5. How to Use – Application Guide

This chapter discusses how Lightstep can be used in application. The usage method will be helpful in special situations.

## 5.1 Dealing with Various Types of Controllers

If you use the “ControllerBase” class, it is easy to create applications for the Ethernet type as well as PCI and RS-232C controller types.

### 5.1.1 ControllerBase Class

The **EthernetController** class, **PciBusController** class, and **Rs232cController** class are all derived from the **ControllerBase** class. If you create an instance for each controller type class and save it in the variable of the **ControllerBase** class, the type of controller can be differentiated.

The process related to sending and receiving commands such as connect, send, receive, anddisconnect is all defined by the **ControllerBase** class, and normal communication can be performed without typecasting.

| **VB Code Example** |
| --- |
| `Dim controller As ControllerBase = New EthernetController()` |
| **C# Code Example** |
| `ControllerBase controller = new EthernetController();` |

### 5.1.2 Connection String

When an instance is stored in the **ControllerBase** class, the connection string used by different controller types can be accepted to connect with the controller. The connection string refers to the character string describing the parameter to connect with the controller. In the **ControllerBase** class, a parameter in character string has been prepared for the **Connect** method. If the setup data for a connection string is created from outside the program, the **Connect** method can be used for connection. It is not necessary to write codes for each controller type.
If the parameters are not correctly specified, an exception, **System.ArgumentException** is raised.

Connection string format is as follows:

```
(parameter name1)=(value);(parameter name2)=(value);
```

The parameter delimiter is a semicolon. Upper and lower case characters are not case sensitive. Spaces are excluded.

The following is a summary of the parameters.

**Ethernet Type**

| Parameter Name | Can Omit (x) | Value when Omitted | Explanation |
|---|---|---|---|
| IpAddress | × | - | IP address of the controller |
| Port | -- | 5003 | TCP port number of the controller |

**RS-232C Type**

| Parameter Name | Can Omit (x) | Value when Omitted | Explanation |
|---|---|---|---|
| Com | × | - | COM port number |
| Baudrate | -- | 9600 | Baud rate |
| DataBits | -- | 8 | Data length |
| StopBits | -- | 1 | Stop bit |
| Parity | -- | None | Parity<br>None: no<br>Even: even number<br>Odd: odd number<br>Mark: always 1<br>Space: always 0<br>(Not case sensitive) |

**PCI Bus Type**

| Parameter Name | Can Omit (x) | Value when Omitted | Explanation |
|---|---|---|---|
| BoardNumber | × | - | Board number |

VB Code Example

```
Dim connectionString As String = "IpAddress=192.168.1.254;"
controller.Connect(connectionString)
```

C# Code Example

```
string connectionString = "IpAddress=192.168.1.254;";
controller.Connect(connectionString);
```

### 5.1.3 IPolling Interface

There is no **StartPolling** method in the **EthernetController** class, but the **PciBusController** and **Rs232cController** classes do. The **IPolling** interface can be used to determine if the **StartPolling** method can be executed.

The **IPolling** interface is implemented in the **PciBusController** class and **Rs232cController** class. The **StartPolling** method is defined in the **IPolling** interface. To determine if you can use **StartPolling**, you need to check if the controller object is compatible with the **IPolling** iterface.

An example of implementation is shown below.

**VB Code Example**

```
Dim controller As ControllerBase
--(Omission)--
If TypeOf controller Is IPolling Then
  CType(controller, IPolling).StartPolling()
End If
```

**C# Code Example**

```
ControllerBase controller;
--(Ommission)--
if (controller is IPolling)
{
   ((IPolling)controller).StartPolling();
}
```

### Save the Polling Command Log

When automatic polling starts with the **StartPolling** method, the "**H**" command is sent to the controller at 100 milisecond intervals and a response for that command is obtained. To prevent the log from becoming huge, by default the polling command is not output to the log.

From the polling result, only when there is some kind of light module completion report, will the content of the "**h**" command in response to the "**H**" command be output to the log.

To output all the polling commands to the log, set the value of the **IPolling** interface's **PollingLogEnabled** to **true**.

## 5.2 Get the Communication Log with the Host Application

The communication log between the controller and Lightstep is stored as a text file. However, the log data can also be obtained from Lightstep through an application. It is useful, for example, when you want to show the communication content on the screen with a testing tool.

To get the log data, implement the **LogMessageRaised** event handler of the controller class, and execute the **GetLogMessages** method from the event handler. Unless you implement the **LogMessageRaised** event handler, you will not be able to get anything even when the **GetLogMessages** method is executed. Here, the automatic log status (value of **xxx.Logger.Enabled**) will not be impacted.

The data that has been obtained will be presented by a **LogEventItem** object array. In the **LogEventItem** object, the log data, the log type, and log occurrence time will be stored.

## 5.3 Unique Functions of Each Controller Type

Functions unique to each controller are explained below.

### 5.3.1 For the Ethernet Type

There are no unique functions for the **EthernetController** class, but the return value of the **SendCommand** and **GetCommand** methods is passed by the instance of the **EthernetCommandInfo** class, which is the derived class of the **CommandInfo** class.

When the **EthernetCommandInfo** class is used, you can get the sequence number of the frame of the sending/receiving command as well as the length of the transmission command.

### 5.3.2 For the PCI Bus Type

In the **PciBusController** class, there is the **GetBoardCount** method that can get the number of controllers inserted in the PC and the **GetInformation** method that can get PCI bus controller information.

With the **GetInformation** method, specify the board number to get the I/O address, slot number, and bus number.

### 5.3.3 For the RS-232C Type

With the **Rs232cController** class, there is the **GetCheckSum** method to perform checksum calculation. This is a protected method. If the method of checksum differs from that of the standard RS-232C controllers, you can create a derived class and override the **GetCheckSum** method and replace with any checksum method.

Also, the return value of the **SendCommand** and **GetCommand** methods is passed by the instance of the **Rs232cComandInfo** class, which is the derived class of the **CommandInfo** class.

When the **Rs232cCommandInfo** class is used, you can get the checksum that was added to the command.

# 6. Distribution of Application

This chapter discusses the distribution method for applications developed by using Lightstep.

## 6.1 Distribution Files

Below are 2 files that can be attached to an application developed using Lightstep. These files are located in the installation folder of Lightstep. The files can be used in both 32/64 bit environments.

| | |
|---|---|
| AioiSystems.Lightstep.dll | DLL of Lightstep. |
| AioiSystems.Lightstep.resources.dll | Resource file for Japanese error messages. It is placed in the “**ja**” folder. |

## 6.2 Placement Location

Put the **AioiSystems.Lightstep.dll** file in the same folder as the **.exe** file of the application. Also in the case of the Japanese application, create a “**ja**” folder in the same folder and put **AioiSystems.Lightstep.resources.dll** inside. (For Japanese application only)

```
¥(AppFolder)
  ¥ja
    AioiSystems.Lightstep.resources.dll
  AioiSystems.Lightstep.dll
  xxx.exe
```

In addtition, by registering in the Global Assembly Cache, you can share and use from multiple applications. As for the registeration method, refer to the MSDN online help as an example.

## 6.3 About Lightstep Version

**AioiSystems.Lightstep.dll** is a strong-named assembly. The application only recognizes the Lightstep version that was used to build it. When multiple Lightstep versions are used, be aware of the version you are using.

To update to a new version of Lightstep used by the host application, redo the project’s reference setup with Visual Studio, and rebuild the project. Also, check if there are any changes in the interface compatibility.

# 7. Migrating from AiPlug

This chapter discusses how to migrate from AiPlug to Lightstep for the existing .NET applications that use AiPlug.

## 7.1 Determine the Migration Direction

With Lightstep, asynchronous communication, which was not available with AiPlug, became possible. To migrate from AiPlug to Lightstep and use asynchronous communication, you need to reevaluate the application design. A program based on synchronous communication and asynchronous communication has a different design concept, so just replacing methods may cause unintended operation.

The synchronous migration can be performed relatively easily. Lightstep has functions similar to the AiPlug functions, and in most cases migration can be accomplished by simply rewriting.

An explanation on the migration of synchronous communication is provided below.

## 7.2 Comparing with AiPlug

The table below shows the AiPlug method and properties that correspond to those of the Lightstep controller class.

| Category | Item | AiPlug | Lightstep Controller Class |
|---|---|---|---|
| Method | Connect | ConnectSocket<br>ConnectPci<br>ConnectRs232c | Connect |
| | Send a command | SendCommand | SendCommand |
| | Receive command manually | GetCommand | GetCommand |
| | Disconnect | CloseController | Close |
| | Start automatic logging | StartLogging | Logger.Enabled = true |
| | End automatic logging | StopLogging | None |
| | Start automatic polling | StartPolling | StartPolling<br>(IPolling interface) |
| | End automatic polling | StopPolling | StopPolling<br>(IPolling interface) |
| Event | Command receive event | DataArrival | CommandReceived |
| | Error event | ErrorRaised | ErrorRaised |

| | | | |
|---|---|---|---|
| Property | Binary mode | BinaryMode | BinaryMode |
| | Timeout time | CommandTimeout | CommandTimeout |
| | "**h**" command conversion | Convert_h | None |
| | Error message of the last error | ErrorDescription | None |
| | Error number of the last error | ErrorNumber | None |
| | Connection status | IsConnected | IsConnected |
| | Polling status | IsPolling | IsPolling<br>(IPolling interface) |

## 7.3 Main Points When Replacing

Below are the main points when simply replacing AiPlug with Lightstep.

(1) Drag the controller class you are going to use from the tool box and put it on the form where AiPlug is placed. (Placing on the form activates automatic synchronization with the main thread.)

(2) Set up the license by using the **SetLicense** method before connecting with the controller.

(3) Similarly, just before connecting with the controller, use the **Logger** property and set up automatic logging. (Replaced from the **StartLogging** method. Note: this is before connecting.)

(4) Replace the AiPlug's **BinaryMode** and **CommandTimeout** properties with the controller class' properties. The default value of the **BinaryMode** property is the opposite of the default value of AiPlug. The default value of the **CommandTimeout** also differs from that of AiPlug.

(5) Replace the connection method.

(6) When AiPlug's **StartPolling** method is used for the PCI bus and RS-232C type controllers, replace it with the controller class' **StartPolling** method.

(7) Replace the **SendCommand** method of AiPlug with the **SendCommand** method of the controller class.

(8) If the **GetCommand** method of AiPlug is used, replace it with the **GetCommand** method of the controller class.

(9) Create a **CommandReceived** event handler and copy the internal process of the **DataArrival** event handler. Where **e.Data** is used to get a command, it should be replaced with the **GetCommand** method.

(10) When **Convert_h** property is used and set to **true**, there is no corresponding function in the controller class, the "**h**" command can be passed with a command. If the process of the command analysis needs to be changed, please consider using the **AddressInfo** class. The "**h**" command and "**t**" command both work with the **AddressInfo** class.

(11) If **IsConnected** property and **IsPolling** property are used, replace them with the **IsConnected** property and **IsPolling** property of the controller class

(12) Replace the disconnect method.

(13) After checking the operation, delete places where there are AiPlug descriptions, and delete the AiPlug control from the form. Also delete the wrapper class (**AiPlugControl, AxAiPlugControl**) which was automatically added with the reference settting.

The above procedure is a general summary of main points. Make sure to thoroughly check if the post migration behavior is the same as that of the pre migration.

## 7.4 Change the Error Process

In AiPlug, the existence of error was determined by the method's return value of **True** or **False**, and the error content was obtained with the **ErrorNumber** property and **ErrorDescription** property. Because Lightstep utilizes the exception mechanism, it is necessary to change the error processing method.

If you want to do simple replacement from AiPlug to Lightstep, change the program on the following points.

- Error process when the connect/disconnect methods are executed.
- Error process when sending commands.
- Error event handling process.

**[Note]**

- No part of this manual may be copied or reproduced in whole or in part without the written permission of AIOI·SYSTEMS CO., LTD.
- The contents of this manual may be revised without prior notice.
- The software specification may be revised without prior notice.

Windows, Windows 10, Windows 8, Windows 7, Windows XP, Windows Vista, Windows Server 2003, Windows Server 2008, .NET Framework are registered trademarks of Microsoft Corporation

**Lightstep 1.0 Programing Guide**
Third Edition

**AIOI•SYSTEMS CO., LTD.**
1-6-8 Omori Kita, Ota-ku, Tokyo 143-0016 JAPAN
Tel: 03-3764-0228 Fax: 03-3764-7520
E-mail: info1@hello-aioi.com
Web Site: http://www.hello-aioi.com/